
Distributed Databases: Database Links and Transparency

In this Lab, we will explore concepts related to distributed databases (DDB) with a specific focus on database links. DDB refers to a database that is spread across multiple nodes, working together as a single logical database. In previous labs, we discovered data partitioning as a technique to manage large data volumes. Now, we will focus on how these partitioned databases can communicate and work together through database links.

Database links are essential components that allow a database to access objects stored in another database. They enable seamless querying, joining, and modifying of data across multiple database instances, creating a unified view of distributed data. This lab will demonstrate how to establish these communication pathways, combine distributed data through global views, and maintain data consistency through distributed transactions.

In this lab, you will:

- Set up database links between sites to enable cross-site queries
- Create global views and synonyms for transparent access to distributed data
- Implement and test distributed transactions across multiple sites

① Useful Notions & Syntaxes

(a) Database Links

Database links allow Oracle databases to access objects in other databases.

i) Creating a Database Link

```
1  -- Private database link (only accessible by the creator)
2  CREATE DATABASE LINK link_name
3  CONNECT TO username IDENTIFIED BY password
4  USING 'connect_string';
5
6  -- Query to view all database links owned by the current user
7  SELECT db_link, username, host
8  FROM user_db_links;
9
10 -- Public database link (accessible by all users)
11 CREATE PUBLIC DATABASE LINK link_name
12 CONNECT TO username IDENTIFIED BY password
13 USING 'connect_string';
```

ii) Using Database Links

```
1  -- Query a remote table
2  SELECT * FROM table_name@link_name;
3
```

```

4  -- Join a local table with a remote table
5  SELECT a.column1, b.column2
6  FROM local_table a
7  JOIN remote_table@link_name b ON a.key = b.key;
8
9  -- Insert into a remote table
10 INSERT INTO table_name@link_name VALUES (...);

```

(b) Global Views and Synonyms

i) Creating Global Views

```

1  -- Create a view that combines data from multiple sites
2  CREATE VIEW global_view_name AS
3  SELECT column1, column2, 'Site1' as site FROM table@site1_link
4  UNION ALL
5  SELECT column1, column2, 'Site2' as site FROM table@site2_link
6  UNION ALL
7  SELECT column1, column2, 'Site3' as site FROM table@site3_link;

```

ii) Creating Synonyms

```

1  -- Create a synonym for easier access to a remote object
2  CREATE [PUBLIC] SYNONYM synonym_name FOR
   [owner.]object_name[@database_link_name];

```

(c) Distributed Transactions

Oracle automatically handles distributed transactions using a two-phase commit protocol to ensure atomicity.

```

1  -- Example of a distributed transaction
2  BEGIN
3  -- Operation on local table
4  UPDATE local_table SET column1 = value1 WHERE condition;
5
6  -- Operation on remote table
7  UPDATE remote_table@site_link SET column1 = value2 WHERE
   condition;
8
9  -- The commit initiates a two-phase commit protocol
10 COMMIT;
11 END;
12 /

```

② Distribution Strategy and Implementation Tasks

This section provides a series of guided questions and tasks to help you implement a distributed database system using database links.

(a) Distribution Strategy Understanding

The IMDB database has been horizontally partitioned across three separate database instances:

Site 1: Movies (MOVIES_SITE)

- Contains all records where showType = 'movie'
- Related Staff, Principals, Episode and Rating data for movies is also stored here

Site 2: TV Content (TV_SITE)

- Contains all TV-related content, including: tvEpisode, tvSeries, tvMiniSeries, tvMovie, tvShort, and tvSpecial.
- Contains all related TV staff, principal cast/crew, episode, and rating information

Site 3: Other Content (OTHER_SITE)

- Contains remaining content types: short, video, and videoGame.
- Contains all related data for these content types

Global Access Layer

- We'll implement a global administrator user that creates database links, views, and synonyms
- This layer provides transparent access to all data across the three sites
- End users will be able to query distributed data without needing to know which site contains which data

Q1 Draw a diagram illustrating the three sites and how they connect to form a distributed database system. Include the global access layer in your diagram.

(b) Implementation Tasks: Setting Up Site Users

To begin implementing a distributed database system, we first need to set up the necessary database users:

```

1  -- Connect as SYSDBA
2  sqlplus / as sysdba
3
4  -- Create users for each site
5  ALTER SESSION SET "_ORACLE_SCRIPT"=TRUE;
```

Q1 Complete the SQL commands to create users for each site (MOVIES_SITE, TV_SITE, OTHER_SITE, and GLOBAL_ADMIN) with appropriate passwords and quotas.

Q2 Write the SQL commands to grant the necessary privileges to each user.

```

1  -- Grant necessary privileges
2  GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE
   PROCEDURE, CREATE DATABASE LINK TO movies_site;
```

Q3 Write the SQL commands to grant SELECT privileges on IMDBMANAGER tables to the site users.

```

1  -- Grant select on IMDBMANAGER tables to site users
2  GRANT SELECT ON IMDBMANAGER.SHOW TO movies_site, tv_site,
   other_site, global_admin;
```

(c) Implementation Tasks: Creating and Populating Tables

Before setting up database links, we need to create and populate the tables at each site with data from IMDBMANAGER.

Q1 Write the SQL commands to connect to MOVIES_SITE and create the Show table with appropriate columns. Then write the INSERT statement to populate it with movie data from IMDBMANAGER (IMDBMANAGER.SHOW).

Q2 Repeat the table creation and data population for TV_SITE and OTHER_SITE. Ensure you're selecting the correct data for each site based on the showType values described in the distribution strategy.

Q3 After populating the tables, how would you verify that the data has been distributed correctly across the three sites? Write queries for each site to count the number of records and verify there is no overlap.

(d) **Implementation Tasks: Creating Private Database Links Between Sites**

Private database links are only accessible by the user who creates them. We'll create these links to allow each site to access the other sites.

Q1 Write the SQL commands to connect to MOVIES_SITE and create private database links to the other two sites.

Q2 Write similar commands for TV_SITE and OTHER_SITE to create their respective private links.

Q3 After creating the private links, write test queries from each site to verify that the links are working properly.

(e) **Implementation Tasks: Setting Up Public Database Links**

Public database links are accessible by all users in the database. We'll create these to allow global views that can be accessed by anyone.

Q1 Connect as SYSDBA and write the SQL commands to create public database links to all three sites.

The key differences between private and public database links are:

- Private links can only be used by the user who created them (e.g., only movies_site can use the tv_link it created)
- Public links can be used by any user in the database with appropriate privileges

(f) **Implementation Tasks: Creating Global Views for Transparent Access**

Q1 Connect as GLOBAL_ADMIN and write the SQL command to create a global view named GLOBAL_SHOWS that combines data from all three sites.

Q2 Write a query to test the global view. Limit the results to show only 10 records.

Q3 Create additional global views for other tables in the database (e.g., Staff, Principals).

(g) **Implementation Tasks: Creating Public Synonyms**

Synonyms provide a simpler way to access objects across schemas when working in a single database instance.

Q1 Connect as GLOBAL_ADMIN and write SQL commands to create public synonyms for the global views.

Q2 Create synonyms for direct access to remote tables.

Q3 Write test queries using the synonyms to verify they work correctly.

(h) **Implementation Tasks: Implementing a Distributed Transaction**

Q1 Connect as GLOBAL_ADMIN and write a distributed transaction that updates show titles from the year 2020 to uppercase format in both the movies database and the TV database.

(i) **Implementation Tasks: Testing Distributed Queries**

Q1 Connect to MOVIES_SITE and write a query to find the total number of shows by site using the public synonym for the global view.

Q2 Write a query to find the total number of shows by type across all sites using the public links.

Q3 Write a query to find all shows from the 1990s (startYear between 1990 and 1999) from both Movies and TV sites, but not from the Other site, using private database links. Connect as movies_site.

Q4 Connect as tv_site and find all shows that have the same title in both the TV site and Movies site.

3 Homework

By the end of each Lab, a report should be done, that contains all the details of the elaborate work during the Lab session. The report is going to be submitted later on. **You can work in groups of three maximum.**